

I. Operatorul *DIVISION*.

II. Variabile de substituție

I. Implementarea operatorului *DIVISION* în *SQL*

Diviziunea este o operație binară care definește o relație ce conține valorile atributelor dintr-o relație care apar **în toate** valorile atributelor din cealaltă relație.

Operatorul *DIVISION* este legat de cuantificatorul universal ($\forall$) care nu există în *SQL*. Cuantificatorul universal poate fi însă simulat cu ajutorul cuantificatorului existențial ($\exists$) utilizând relația:

$$\forall x P(x) \equiv \neg \exists x \neg P(x).$$

Prin urmare, operatorul *DIVISION* poate fi exprimat în *SQL* prin succesiunea a doi operatori *NOT EXISTS*. Alte modalități de implementare a acestui operator vor fi prezentate în exemplul de mai jos.

Extindem diagrama *HR* cu o nouă entitate, *PROJECT*, și o nouă asociere: „angajat lucrează în cadrul unui proiect”, între entitățile *EMPLOYEES* și *PROJECT*. Aceasta este o relație *many-to-many*, care va conduce la apariția unui tabel asociativ, numit *WORKS_ON*.

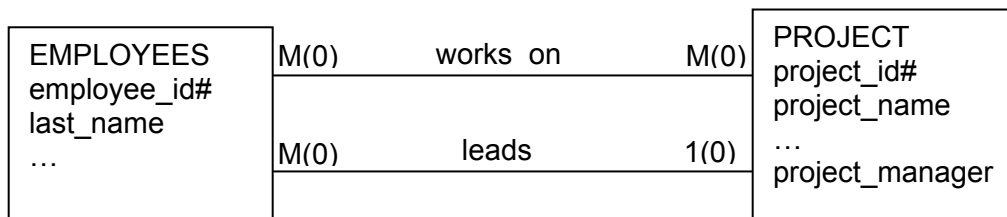
O altă asociere între entitățile *EMPLOYEES* și *PROJECT* este „angajat conduce proiect”. Aceasta este o relație *one-to-many*.

Noile tabele au următoarele scheme relaționale:

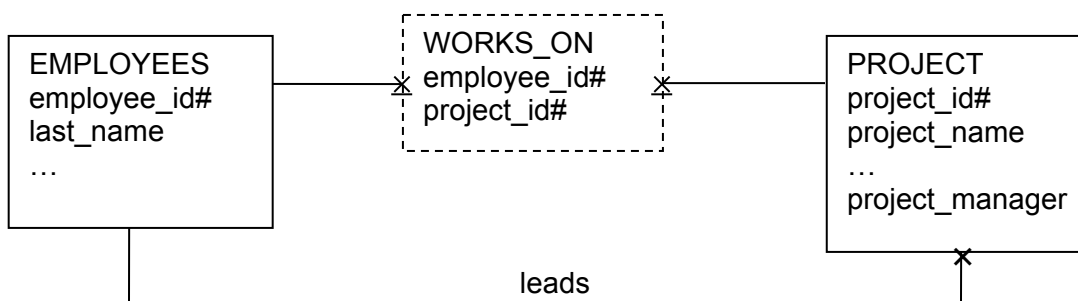
- 1) *PROJECT*(project_id#, project_name, budget, start_date, deadline, delivery_date, project_manager)
 - project_id reprezintă codul proiectului și este cheia primară a relației *PROJECT*
 - project_name reprezintă numele proiectului
 - budget este bugetul alocat proiectului
 - start_date este data demarării proiectului
 - deadline reprezintă data la care proiectul trebuie să fie finalizat
 - delivery_date este data la care proiectul este livrat efectiv
 - project_manager reprezintă codul managerului de proiect și este cheie externă. Pe cine referă această coloană ? Ce relație implementează această cheie externă?
- 2) *WORKS_ON*(project_id#, employee_id#, start_date, end_date)
 - cheia primară a relației este compusă din attributele employee_id și project_id.

Scriptul pentru crearea noilor tabele și inserarea de date în acestea este *hr_project.sql*.

Diagrama entitate-relație corespunzătoare modelului *HR* va fi extinsă, pornind de la entitatea *EMPLOYEES*, astfel:



Partea din diagrama conceptuală corespunzătoare acestei extinderi a modelului este următoarea:



Exemplu: Să se obțină codurile salariaților atașați **tuturor** proiectelor pentru care s-a alocat un buget egal cu 10000.

Metoda 1 (utilizând de 2 ori *NOT EXISTS*):

```

SELECT      DISTINCT employee_id
FROM        works_on a
WHERE NOT EXISTS
  (SELECT    1
   FROM      project p
   WHERE     budget=10000
   AND NOT EXISTS
     (SELECT  'x'
      FROM    works_on b
      WHERE   p.project_id=b.project_id
      AND     b.employee_id=a.employee_id));
  
```

Metoda 2 (simularea diviziunii cu ajutorul funcției *COUNT*):

```

SELECT      employee_id
FROM        works_on
WHERE       project_id IN
  (SELECT    project_id
   FROM      project
   WHERE     budget=10000)
GROUP BY    employee_id
HAVING      COUNT(project_id)=
  (SELECT    COUNT(*)
   FROM      project
   WHERE     budget=10000);
  
```

Metoda 3 (operatorul *MINUS*):

```

SELECT DISTINCT employee_id
FROM  works_on
MINUS
SELECT employee_id from
  ( SELECT employee_id, project_id
    FROM (SELECT employee_id FROM works_on) t1,
          (SELECT project_id FROM project WHERE budget=10000) t2
    MINUS
  )
  
```

```
SELECT employee_id, project_id FROM works_on
) t3;
```

Metoda 4 ($A \text{ include } B \Rightarrow B \setminus A = \emptyset$):

```
SELECT      DISTINCT employee_id
FROM        works_on a
WHERE NOT EXISTS (
    (SELECT    project_id
    FROM      project p
    WHERE     budget=10000)
    MINUS
    (SELECT    project_id
    FROM      project p, works_on b
    WHERE     p.project_id=b.project_id
    AND       b.employee_id=a.employee_id));
```

II. Variabile de substituție

- Variabilele de substituție sunt utile în crearea de comenzi/script-uri dinamice (care depind de niste valori pe care utilizatorul le furnizează la momentul rularii).
- Variabilele de substituție se pot folosi pentru stocarea temporară de valori, transmiterea de valori între comenzi SQL etc. Ele pot fi create prin:
 - **Prefixarea cu &** (indica existența unei variabile într-o comandă SQL, dacă variabila nu există, este creată).
 - **Prefixarea cu &&** (indica existența unei variabile într-o comandă SQL, dacă variabila nu există, este creată). Deosebirea față de & este că, dacă se folosește &&, atunci referirea ulterioară cu & sau && nu mai cere ca utilizatorul să introducă de fiecare dată valoarea variabilei. Este folosită valoarea dată la prima referire.

Variabilele de substituție pot fi eliminate cu ajutorul comenzii *UNDEF[INE]*

Observatii:

- Variabilele de tip *DATE* sau *CHAR* trebuie să fie incluse între apostrofuri în comandă *SELECT*.
- După cum le spune și numele, variabilele de substituție înlocuiesc/substituie în cadrul comenzii SQL variabila respectivă cu sirul de caractere introdus de utilizator.
- Variabilele de substituție pot fi utilizate pentru a înlocui la momentul rularii:
 - condiții *WHERE*;
 - clauza *ORDER BY*;
 - expresii din lista *SELECT*;
 - nume de tabel;
 - o întreaga comandă SQL;
- Odată definită, o variabilă rămâne până la eliminarea ei cu o comandă *UNDEF* sau până la terminarea sesiunii SQL*Plus respective.
- Comanda *SET VERIFY ON | OFF* permite afișarea sau nu a comenzii înainte și după înlocuirea variabilei de substituție.

Exerciții:

1. Să se listeze informații despre angajații care au lucrat în toate proiectele demarate în primele 6 luni ale anului 2006. Implementați toate variantele.
2. a) Să se obțină numele angajaților care au lucrat **cel puțin** pe aceleași proiecte ca și angajatul având codul 200.
Obs: Incluziunea dintre 2 mulțimi se testează cu ajutorul proprietății „ $A \text{ inclus în } B \Rightarrow A-B = \emptyset$ ”. Cum putem implementa acest lucru în SQL?
 Pentru rezolvarea exercițiului, trebuie selectați angajații pentru care este vidă lista proiectelor pe care a lucrat angajatul 200 mai puțin lista proiectelor pe care au lucrat acei angajați.
- b) Să se obțină numele angajaților care au lucrat **cel mult** pe aceleași proiecte ca și angajatul având codul 200.
- c) Să se obțină angajații care au lucrat pe aceleași proiecte ca și angajatul având codul 200.
Obs: Egalitatea între două mulțimi se testează cu ajutorul proprietății „ $A=B \Rightarrow A-B=\emptyset$ și $B-A=\emptyset$ ”.
3. **Pentru fiecare țară**, să se afișeze numărul de angajați din cadrul acesteia.
4. Sa se afișeze codul, numele, salariul si codul departamentului din care face parte pentru un angajat al carui cod este introdus de utilizator de la tastatura.
5. Sa se afiseze numele, codul departamentului si salariul anual pentru toti angajatii care au un anumit job.
6. Sa se afiseze numele, codul departamentului si salariul anual pentru toti angajatii care au fost angajati dupa o anumita data calendaristica.
7. Sa se afiseze o coloana aleasa de utilizator, dintr-un tabel ales de utilizator, ordonand dupa aceeasi coloana care se afiseaza. De asemenea, este obligatorie precizarea unei conditii WHERE.
8. Să se citească două date calendaristice de la tastatură și să se afișeze zilele dintre aceste două date.

Exemplu: Dacă se introduc datele 1-apr-2008 și 14-apr-2008, rezultatul cererii va fi:

01-apr-2008
 02-apr-2008
 ...
 14-apr-2008

Modificați cererea anterioară astfel încât să afișeze doar zilele lucrătoare dintre cele două date calendaristice introduse.